

Rails Models & User Management

ActiveRecord, Associations, and Devise Authentication

1. ActiveRecord ORM & Migrations

ActiveRecord is the M (Model) in MVC. It is an Object-Relational Mapping (ORM) system that maps database tables to Ruby classes.

Migrations

Migrations are a DSL for managing your database schema over time. Instead of writing SQL, you write Ruby code.

```
# Generate a migration
rails generate migration CreatePosts title:string body:text user:references

# The generated migration file
class CreatePosts < ActiveRecord::Migration[7.0]
  def change
    create_table :posts do |t|
      t.string :title
      t.text :body
      t.references :user, null: false, foreign_key: true
      t.timestamps
    end
  end
end
```

2. Validations and Callbacks

Validations

Validations ensure that only valid data is saved to your database.

```
class Post < ApplicationRecord
  validates :title, presence: true, length: { minimum: 5 }
  validates :body, presence: true
end
```

Callbacks

Callbacks are methods that get called at certain moments of an object's life cycle (e.g., before saving, after creating).

```
class User < ApplicationRecord
  before_save :normalize_email

  private
  def normalize_email
    self.email = email.downcase.strip
  end
end
```

3. Model Relationships (Associations)

Associations allow you to define the connection between different models.

One-to-Many (User & Posts)

```
# app/models/user.rb
class User < ApplicationRecord
  has_many :posts, dependent: :destroy
end

# app/models/post.rb
class Post < ApplicationRecord
  belongs_to :user
end
```

Many-to-Many (Post & Tags)

```
# Using has_many :through
class Post < ApplicationRecord
  has_many :post_tags
  has_many :tags, through: :post_tags
end
```

4. User Management with Devise

Devise is the industry standard for authentication in Rails. It provides sign-up, login, password reset, and more.

Setup Steps

1. Add **gem 'devise'** to Gemfile and run **bundle install**.
2. Run **rails generate devise:install**.
3. Generate User model: **rails generate devise User**.
4. Run **rails db:migrate**.

Restricting Access

In your controllers, you can ensure a user is logged in before they can create a post:

```
class PostsController < ApplicationController
  before_action :authenticate_user!, except: [:index, :show]

  def create
```

```
@post = current_user.posts.build(post_params)
if @post.save
  redirect_to @post
end
end
end
```

5. Implementation: Simple Blog Example

Combining all concepts to build a simple blog where users own posts.

Step 1: The Form with Validations

```
# app/views/posts/_form.html.erb
<%= form_with(model: post) do |f| %>
  <% if post.errors.any? %>

    <%= pluralize(post.errors.count, "error") %> prohibited this

    <% post.errors.full_messages.each do |message| %>

      <%= message %>

    <% end %>

  <% end %>

  <%= f.label :title %>
  <%= f.text_field :title %>

  <%= f.label :body %>
  <%= f.text_area :body %>

  <%= f.submit %>
<% end %>
```

Step 2: Sign-up/Login Links

```
# app/views/layouts/application.html.erb
<% if user_signed_in? %>
  Logged in as: <%= current_user.email %>
  <%= link_to 'Logout', destroy_user_session_path, data: { "turbo-method": :delete } %>
<% else %>
```

```
<%= link_to 'Login', new_user_session_path %>
<%= link_to 'Sign Up', new_user_registration_path %>
<% end %>
```

Key Takeaway: Rails follows "Convention over Configuration." By naming your associations correctly (e.g., **has_many :posts**), Rails automatically knows how to write the SQL joins and handle the database foreign keys.